

Project Report: Week 3

Summary & Technical Revision

1. Machine Learning Classifications

Machine Learning was structured into three distinct structural paradigms based on how an agent experiences data:

- **Supervised Learning:** The model maps an input to a specific output using labeled datasets under a guided framework ("Train Me!").
 - **Unsupervised Learning:** The model operates on unlabeled data to naturally discover hidden structural groupings and mathematical patterns ("Self-Sufficient").
 - **Reinforcement Learning:** An agent interacts dynamically with an environment, learning an optimal policy via a trial-and-error reward and penalty system ("Hit & Trial").
-

2. Supervised Learning: Regression Models

Used when the target output space is a continuous numeric value (e.g., historical financial prices).

A. Linear Regression (Simple & Multiple)

Fits a straight linear path to minimize the residual errors between actual observations and historical predictions.

$$Y_i = \beta_0 + \beta_1 X_i + \epsilon$$

```
from sklearn.linear_model import LinearRegression
from sklearn import metrics

# 1. Instantiate and Fit Model
reg_model = LinearRegression()
reg_model.fit(X_train, Y_train)

# 2. Extract Attributes & Evaluate
coefficients = reg_model.coef_
```

```
intercept = reg_model.intercept_  
mse = metrics.mean_squared_error(Y_test, Y_pred)
```

Code Term Breakdown:

- `LinearRegression()`: Initializes an Ordinary Least Squares (OLS) linear model blueprint.
- `.fit(X_train, Y_train)`: Optimizes the slope parameters and intercept by minimizing the Mean Squared Error (MSE) cost function, often handled via **Gradient Descent** convergence.
- `model.coef_` / `model.intercept_`: Extracts the trained coefficients and the structural cross-axis intercept point from memory.
- `metrics.mean_squared_error(...)`: Measures performance by taking the average squared difference between the true test targets and your model's predicted outputs.
- **Multicollinearity & VIF**: High correlation among independent features inflates coefficient variance, making estimates unstable. The **Variance Inflation Factor (VIF)** is monitored; a heuristic value of $VIF > 10$ mandates dropping the highly collinear feature.

B. Random Forest Regressor

An ensemble technique that constructs an array of independent decision trees via Bootstrap Aggregating (**Bagging**) and random feature selection, averaging their separate outputs to output robust predictions and prevent overfitting.

3. Supervised Learning: Classification Models

Used to map inputs to discrete, categorical target boundaries.

```
from sklearn.linear_model import LogisticRegression  
  
# 1. Classification Fitting  
clf = LogisticRegression(random_state=0)  
clf.fit(X, y)  
  
# 2. Probability Scores & Class Evaluation
```

```
predicted_class = clf.predict(X[:2, :])
probabilities = clf.predict_proba(X[:2, :])
```

Code Term Breakdown:

- `LogisticRegression()` : Fits a categorical classifier by mapping a linear combination of features through a **Logit / Sigmoid Function**, bounding values between 0 and 1 to express a class probability.
- `.predict()` : Assigns the final hard categorical boundary class label (e.g., 0 or 1) based on the underlying probability thresholds.
- `.predict_proba()` : Outputs the explicit raw matrix array containing confidence weights for every possible target class.
- **Support Vector Machines (SVM)**: Another prominent classification concept covered that optimizes boundary margins by locating separating hyperplanes defined by maximum geometric distances from nearest coordinate pointers (**Support Vectors**).

4. Unsupervised Learning: Clustering

Unsupervised algorithms look for structural geometry, distance anomalies, or spatial associations within unlabeled datasets.

```
from sklearn.cluster import KMeans

# 1. Initialize and Fit K-Means
kmeans = KMeans(n_clusters=2, random_state=0, n_init="auto")
kmeans.fit(X)

# 2. Extract Spatial Cluster Properties
assigned_labels = kmeans.labels_
centroids = kmeans.cluster_centers_
```

Code Term Breakdown:

- `KMeans(n_clusters=2)` : Sets up a clustering routine targeting exactly K=2 partitions.

- `n_init="auto"` : Configures how many times the algorithm will independently re-initialize its random cluster centroids to prevent converging on poor local minima.
- `kmeans.labels_` : Returns an integer index array mapping exactly which cluster identity bucket each training data point was grouped into.
- `kmeans.cluster_centers_` : Outputs the localized coordinates of the converged mathematical centroids representing the average center of each spatial cluster group.
- **The Elbow Method:** Used to evaluate the optimum value of K by plotting the Within-Cluster Sum of Squares (WSS) error metrics against an ascending sequence of cluster counts, finding the structural "elbow" bend where diminishing returns on error reduction occur.